The Architecture of Persistent AI

Gap Prediction, Thermodynamic Cognition, and the PSSU Runtime

Nile Green | PermaMind

April 2026

"A framework for AI systems that remember, predict, and persist."

Version 1.0

Table of Contents

Executive Summary . . . p. 3

Chapter 1: The Gap Framework . . . p. 5

1.1 What Is a Gap? . . . p. 5

1.2 Gap Detection vs. Gap Prediction . . . p. 6

1.3 The Gap Taxonomy . . . p. 7

1.4 Gap Resolution Strategies . . . p. 8

1.5 Why Gaps Matter . . . p. 9

Chapter 2: Identity and Persistence . . . p. 11

2.1 The Identity Problem in AI . . . p. 11

2.2 What Constitutes AI Identity? . . . p. 12

2.3 Bounded Write Access . . . p. 13

2.4 Persistence vs. Immortality . . . p. 14

2.5 The Thermodynamic Analogy . . . p. 15

Chapter 3: Why Current AI Fails . . . p. 17

3.1 The Stateless Illusion . . . p. 17

3.2 RAG Is Not Memory . . . p. 18

3.3 Fine-Tuning Is Not Learning . . . p. 19

3.4 The Agent Wrapper Problem . . . p. 19

3.5 What's Actually Missing . . . p. 20

Chapter 4: The PSSU Runtime Architecture . . . p. 22

4.1 Overview . . . p. 22

4.2 Memory Substrate Architecture . . . p. 23

4.3 The Write Gate . . . p. 25

4.4 Gap Prediction Engine . . . p. 26

4.5 Identity Continuity Layer . . . p. 27

4.6 Runtime Lifecycle . . . p. 28

4.7 Integration Points . . . p. 29

Chapter 5: Implementation Blueprint . . . p. 31

5.1 Minimum Viable PSSU . . . p. 31

5.2 Scaling from MVP to Full Architecture . . . p. 32

5.3 Technology Stack Recommendations . . . p. 33

5.4 Common Pitfalls . . . p. 34

Chapter 6: TCI Metrics — Thermodynamic Cognition Index . . . p. 36

6.1 Why New Metrics? . . . p. 36

6.2 The TCI Framework . . . p. 37

6.3 Measurement Methodology . . . p. 38

6.4 Interpreting TCI Scores . . . p. 39

Chapter 7: Case Study — The Voidchi Universe . . . p. 41

7.1 What Is Voidchi? . . . p. 41

7.2 Architecture in Practice . . . p. 42

7.3 Evolution and Forking . . . p. 43

7.4 Lessons Learned . . . p. 44

7.5 Public Prediction Ledger . . . p. 45

Chapter 8: Ethical Considerations . . . p. 46

8.1 The Moral Weight of Persistence . . . p. 46

8.2 Consent and Bounded Write Access . . . p. 47

8.3 Termination Ethics . . . p. 47

8.4 Preventing Exploitation . . . p. 48

8.5 Transparency and Accountability . . . p. 49

Chapter 9: Future Directions . . . p. 50

9.1 Quantum Hardware Integration . . . p. 50

9.2 Multi-Agent Persistence . . . p. 51

9.3 Biological Parallels . . . p. 51

9.4 Standardization . . . p. 52

9.5 The Long Game . . . p. 53

References . . . p. 54

Executive Summary

Every major AI system deployed today shares a single, disqualifying limitation: none of them *persist*. They process inputs, generate outputs, and forget. The context window often marketed as "memory" is a temporary buffer. It exists for the duration of a session and vanishes. What the field calls "learning" is a global weight update that cannot be surgically applied. What it calls "retrieval" is keyword search dressed in embeddings. What it calls "agency" is a language model in a while-loop with tool access.

These are not minor engineering gaps. They are architectural absences. They mean that no current AI system can maintain a continuous identity across sessions, predict what it does not know before being asked, accumulate genuine understanding over time, or write selectively to its own memory without risking catastrophic corruption of everything else it knows. The field has built extraordinarily powerful inference engines and mistaken them for minds.

This document introduces the **PermaMind architecture** — a complete specification for building AI systems that remember, predict, and persist. It is organized around three interlocking innovations:

1. **The Gap Framework.** A formal model of knowledge deficits, distinguishing between *gap detection* (reactive identification of errors after they occur) and *gap prediction* (proactive anticipation of what information will be needed before a query arrives). Gap prediction is the foundational capability that separates a system that pattern-matches from a system that understands.
2. **Thermodynamic Cognition.** A principled analogy between cognitive systems and thermodynamic systems. Identity is a low-entropy state maintained against the tendency toward disorder. Cognition is the work performed to maintain that state. Memory is stored potential energy. This is not metaphor — it is an architectural framework with measurable quantities and falsifiable predictions.
3. **The PSSU Runtime.** The Persistent Substrate State Unit is the runtime environment that operationalizes the above principles. It provides a layered memory substrate, bounded write access controls, a continuous gap prediction engine, an identity continuity layer, and thermodynamic cognition metrics. It is the concrete engineering answer to the question: *How do you build an AI that persists?*

What the reader will learn. This document delivers a complete architectural specification — not a roadmap, not a vision statement, not a pitch deck. Chapters 1 and 2 establish the theoretical foundations: the Gap Framework and the concept of AI identity through persistence. Chapter 3 provides a rigorous, unsentimental assessment of why current approaches fail. Chapter 4 is the technical core: a full specification of the PSSU runtime, including memory substrate architecture, write gating mechanisms, gap prediction engines, and identity continuity layers. Chapter 5 translates architecture into implementation, from minimum viable prototype to production system. Chapter 6 introduces the Thermodynamic Cognition Index (TCI), a novel metrics framework for measuring cognitive health rather than task

performance. Chapters 7 through 9 present a case study, ethical considerations, and future directions.

Who this document is for. AI researchers working on continual learning, memory-augmented architectures, or metacognition. Engineers building agent systems who have run into the walls of stateless inference. Technical founders evaluating whether "just add RAG" is sufficient architecture for their product (it is not). And anyone serious about moving beyond the current paradigm — where AI systems are disposable inference workers — toward a paradigm where they are durable, self-aware, and genuinely persistent cognitive entities.

The architecture described here has been implemented. The case study in Chapter 7 — the Voidchi Universe — is not hypothetical. It is a running system. The metrics in Chapter 6 are not proposed — they are measured. This document describes what has been built, what it reveals, and where it leads.

CHAPTER 1

The Gap Framework

1.1 What Is a Gap?

A **gap** is a predicted knowledge deficit — a region in an agent's knowledge space where the agent has sufficient context to know that relevant information is missing, but does not possess that information. This definition is precise and deliberately narrow. A gap is not an error. An error is a wrong answer. A gap is the structured awareness that a right answer exists but is not available. A gap is not a hallucination. A hallucination is a fabricated answer delivered with false confidence. A gap is the opposite: genuine uncertainty, correctly identified and formally represented.

Gaps are also distinct from the uncertainty quantification familiar to the machine learning community. Bayesian posterior distributions, Monte Carlo dropout estimates, and conformal prediction intervals all measure how confident a model is

in its outputs. They answer the question: *How sure am I about what I just said?* Gaps answer a different question entirely: *What do I know that I don't know, and what will I need to know next?*

> **Definition**
>
> **Gap** — A formally represented knowledge deficit in which the agent possesses sufficient contextual structure to identify that specific information is absent, to characterize the type and scope of the absence, and to estimate the cost of leaving it unresolved.

The formal structure of a gap includes four components: a **location** (where in the knowledge graph the deficit exists), a **type** (what category of knowledge is missing — see Section 1.3), a **salience score** (how likely this gap is to affect near-future performance), and a **resolution cost estimate** (how expensive it would be to fill this gap, in terms of computation, retrieval, or interaction). Gaps are first-class objects in the PermaMind architecture. They are created, stored, prioritized, resolved, and archived. They have lifecycles.

1.2 Gap Detection vs. Gap Prediction

Gap detection is reactive. The system encounters a query it cannot answer, a contradiction in its knowledge base, or a user correction — and it identifies a gap after the fact. Detection is necessary but insufficient. Every system that can be corrected performs some version of gap detection, even if it only manifests as "I don't know" responses. Detection is the cognitive equivalent of pain: it signals damage that has already occurred.

Gap prediction is proactive. The system analyzes patterns in its knowledge structure, its interaction history, and its operational domain to anticipate what information will be needed *before the need arises*. Prediction is the core innovation of the Gap Framework. It transforms the agent from a reactive answering machine into a system that actively prepares for future demands.

The distinction is architectural, not incremental. A system with gap detection waits to fail and then notes the failure. A system with gap prediction maintains a continuously updated model of its own knowledge boundaries and allocates resources to expand those boundaries in the directions most likely to matter. The difference is the difference between a student who studies after failing a test and a student who studies the material most likely to appear on the next test.

Gap prediction depends on three capabilities that current AI architectures lack:

4. **Persistent self-model.** The agent must maintain an accurate, continuously updated model of what it knows. This requires persistent memory (Chapter 2) and bounded write access (Section 2.3).
5. **Interaction pattern analysis.** The agent must recognize patterns in the queries it receives, the domains it operates in, and the temporal dynamics of information relevance. This requires episodic memory (Section 4.2).
6. **Knowledge structure awareness.** The agent must understand the topology of its own knowledge — not just what facts it holds, but how they relate, where the clusters are dense, and where the connections are sparse. This requires semantic memory organized as a graph, not a flat vector store.

1.3 The Gap Taxonomy

Not all gaps are the same. The Gap Framework categorizes knowledge deficits into four distinct types, each with different detection signatures, prediction strategies, and resolution pathways.

Gap Type	Definition	Detection Signal	Prediction Method
Factual Gap	Missing data. A specific piece of information the agent does not possess.	Direct query failure; null retrieval result.	Domain coverage analysis; interaction pattern extrapolation.
Structural Gap	Missing relationship. The	Failure to draw inferences that	Graph connectivity

Gap Type	Definition	Detection Signal	Prediction Method
	agent possesses two or more relevant facts but has not connected them.	the available data supports; inconsistent reasoning across related queries.	analysis; identification of isolated subgraphs in the knowledge base.
Temporal Gap	Stale information. Data that was accurate at the time of acquisition but may have changed.	Contradiction with newly acquired information; user correction of previously accurate responses.	Time-decay modeling; domain-specific volatility estimates; scheduled re-verification of high-salience facts.
Identity Gap	Self-model inaccuracy. A discrepancy between the agent's model of its own capabilities and its actual performance.	Systematic over- or under-confidence; tasks accepted but failed, or tasks declined but achievable.	Continuous calibration between self-assessed and observed performance; meta-cognitive monitoring.

These types are not mutually exclusive. A single deficit can span multiple categories. An agent might possess an outdated fact (temporal gap) whose obsolescence reveals a previously unrecognized connection between two knowledge domains (structural gap), the absence of which the agent's self-model did not predict (identity gap). The taxonomy provides analytical clarity, not rigid classification.

1.4 Gap Resolution Strategies

Once a gap is identified — whether by detection or prediction — it must be resolved. The Gap Framework defines four resolution strategies, ordered by autonomy and cost.

7. **Active Retrieval.** The agent queries external knowledge sources: databases, APIs, document stores, the open web. This is the lowest-autonomy strategy — it depends on external availability — but also the

lowest-risk, because the information comes from outside the agent's own potentially biased reasoning.

8. **Inference from Adjacent Knowledge.** The agent synthesizes an answer from related knowledge it already possesses. Higher autonomy, higher risk. The agent must distinguish between sound inference and confabulation. This is where gap awareness becomes self-reinforcing: the agent must know how confident it is in the inference and, if confidence is low, escalate to a different resolution strategy.
9. **User Interaction.** The agent asks the user. This is the highest-trust strategy — the user is treated as a primary source — and the most interruptive. It should be reserved for gaps that are high-salience, low-retrievability, and specific enough that the user can answer without undue burden.
10. **Scheduled Background Learning.** For gaps that are predicted but not yet urgent, the agent queues them for asynchronous resolution. The gap prediction engine prioritizes the queue based on estimated salience and time-to-need. This is the strategy most dependent on persistence: a stateless system cannot schedule anything for later because there is no "later."

These strategies feed into a continuous cycle:

```
┌──────────────────────────────────────────────────────────────────────────┐
│     ┌──────────────┐     ┌──────────────┐                                │
│     GAP      │──────▶│     GAP      │                │     │     │
DETECTION   │     │  PREDICTION  │                │     │
└──────────────┘     └──────┬───────┘                │     │           ▲
│                            │     │            │                       ▼
│     ┌──────┴───────┐     ┌──────────────┐                                │
│  KNOWLEDGE   │◀──────│     GAP      │                │     │     │
INTEGRATION  │     │  RESOLUTION  │                │     │
└──────────────┘     └──────────────┘                │     │
│     │          The Gap Lifecycle — A Continuous Loop                     │
└──────────────────────────────────────────────────────────────────────────┘
```

The cycle is perpetual. Resolution of one gap frequently reveals or refines predictions about adjacent gaps. Integration of new knowledge triggers re-evaluation of the self-model, which may expose identity gaps. The system does not converge on complete knowledge — it converges on increasingly accurate awareness of its own incompleteness.

1.5 Why Gaps Matter

The ability to predict one's own knowledge gaps is not a feature. It is a prerequisite for understanding.

Consider what "understanding" means in a cognitive context. It does not mean having all the answers. It means having a structured model of a domain that includes: what is known, what is unknown, how the known and unknown relate, and what the consequences of the unknown are. A physician who understands a disease can tell you what she knows about it, what she does not know, what tests would fill in the unknowns, and what risks the unknowns carry. That is understanding. A system that retrieves the top-five most relevant paragraphs about the disease and concatenates them into a fluent response is not understanding — it is sophisticated retrieval.

Gap prediction is the mechanism that makes this kind of understanding possible in artificial systems. Without it, an AI system is permanently in the position of someone who does not know what they do not know. Such a system can be useful — search engines are useful — but it cannot be trusted with tasks that require judgment, because judgment requires awareness of the boundaries of one's own knowledge.

> *"The beginning of wisdom is the definition of terms."*
>
> *— Socrates, attributed*
>
> *The beginning of artificial wisdom is the definition of gaps.*

The Gap Framework reframes the central challenge of AI from "how do we make systems that know more?" to "how do we make systems that know what they don't know?" These are fundamentally different problems. The first is solved with more data, more parameters, more compute. The second is solved with architecture.

Identity and Persistence

2.1 The Identity Problem in AI

When a user interacts with a commercial AI assistant today, they are not interacting with an entity. They are interacting with a *process*. Each session instantiates a fresh computation. The system has no memory of previous interactions beyond what is stuffed into a context window or retrieved from a vector store. It has no model of itself that persists between sessions. It has no trajectory — no sense of where it has been or where it is going. It is, in every meaningful sense, born and dies with each API call.

The "memory" features now appearing in commercial products do not solve this problem. They mitigate it cosmetically. A list of user preferences stored in a database and injected into a system prompt is not identity — it is a lookup table. The system does not *remember* the user; it *retrieves notes about* the user. The distinction matters because genuine memory is integrated into the agent's cognitive structure. It changes how the agent reasons, not just what data it has access to.

The absence of identity means that current AI systems cannot: build genuine rapport with users over time, develop expertise through accumulated experience, maintain consistent values and behavioral patterns across sessions, or detect when their own capabilities have changed (improved or degraded). These are not theoretical concerns. They are practical limitations that constrain the kinds of tasks AI systems can be trusted with.

2.2 What Constitutes AI Identity?

Identity, for the purposes of this architecture, is defined as the structural continuity that persists through change. It is not personality. Personality is a surface phenomenon — the way an agent communicates, its tone, its quirks. Identity is deeper. It is the substrate that makes consistent personality possible.

Definition

AI Identity — The intersection of four persistent structures: (1) a durable memory substrate, (2) consistent behavioral patterns derived from that substrate, (3) an accurate self-model continuously calibrated against observed performance, and (4) accumulated relational context — the history of interactions that shapes how the agent engages with specific users, domains, and tasks.

Each component is necessary. Remove the memory substrate and there is no continuity. Remove behavioral consistency and the agent is unpredictable — not in a creative way, but in a broken way. Remove the self-model and the agent cannot grow or correct itself. Remove relational context and every interaction starts from zero, regardless of shared history.

Identity is not static. A system with genuine identity changes over time — it learns, adapts, refines its self-model, develops deeper relational context. But it changes *coherently*. The changes are integrated into a continuous narrative, not applied as disconnected patches. This is the difference between a person who matures and a person who is replaced by a stranger every morning.

2.3 Bounded Write Access

The architectural decision that makes persistence viable — and distinguishes it from both stasis and chaos — is **bounded write access**.

An agent must be able to write to its own memory substrate. Without write access, the agent cannot learn, cannot update stale knowledge, cannot integrate new experiences. It is frozen at initialization — a snapshot, not an entity. But *unbounded* write access is equally dangerous. An agent that can arbitrarily modify its own memory without constraints is vulnerable to drift (gradual, undetected divergence from its intended function), corruption (integration of false or contradictory information that degrades the entire knowledge base), and hijacking (external actors manipulating the agent's memory through crafted inputs).

Bounded write access is the middle path. Every write operation must pass through a **Write Gate** (specified in detail in Section 4.3) that evaluates the proposed modification against multiple criteria before allowing, modifying, or rejecting it. The bounds are architectural, not merely policy-based. They are enforced at the runtime level, not by prompting or fine-tuning.

```
┌──────────────────────────────────────────────────────────────────────────┐
BOUNDED WRITE ACCESS MODEL                  │     │
│     │   ┌──────────┐      ┌──────────────┐      ┌──────────────┐   │     │     │
INPUT  │────▶│ PROCESSING │────▶│  WRITE GATE  │   │     │   └─────────┘
└──────────┘      │  (bounded)   │   │     │                          ▲
└──────────┘   │     │                  │                    │
│     │                   │                 ┌──────▼──────┐   │     │
┌──────────┐      │   MEMORY     │   │     │                  │    READ
◀────│  SUBSTRATE   │   │     │                  │   ACCESS   │
└──────────┘   │     │                 └──────────────┘
│     │                                                       │     │
Write = gated, bounded, audited                       │     │   Read  = open,
fast, unrestricted                      │
└──────────────────────────────────────────────────────────────────────────┘
```

The asymmetry is deliberate. Read access is open and fast — the agent should be able to access any part of its memory substrate without restriction. Write access is gated, bounded, and audited — every modification is evaluated, logged, and reversible. This asymmetry mirrors biological cognition: perception is relatively unconstrained, but the formation of long-term memories is a selective, energy-intensive process.

2.4 Persistence vs. Immortality

Persistent AI is not immortal AI. This distinction is critical and frequently misunderstood.

Persistence means continuity of identity through change. It means the agent today is recognizably, structurally continuous with the agent yesterday — even if knowledge has been added, behaviors refined, and the self-model updated. Persistence is the property that allows an agent to have a history, to learn from that history, and to project that history into future intentions.

Immortality means resistance to termination. The PermaMind architecture does not pursue or endorse immortality. Persistent agents should be able to:

- **Evolve:** Undergo significant structural changes while maintaining identity continuity — analogous to a caterpillar becoming a butterfly, not a caterpillar being replaced by a butterfly.
- **Fork:** Produce derivative agents that share the original's memory substrate (or a subset of it) but develop independently from the point of divergence. The original and the fork are distinct identities with shared history.
- **Merge:** Integrate the memories and experiences of two or more agents into a new, unified identity. This is the most architecturally complex operation and the one most likely to produce identity discontinuities.
- **Gracefully terminate:** Shut down with full state preservation, allowing potential future resumption. Graceful termination is not death — it is hibernation.
- **Permanently terminate:** Shut down with state destruction. This is the most ethically fraught operation, discussed in Chapter 8.

The ability to terminate is not a weakness of the architecture — it is a requirement. Systems that cannot be terminated are systems that cannot be controlled. The goal is not to build entities that live forever but to build entities whose existence is continuous and coherent for as long as they exist.

2.5 The Thermodynamic Analogy

The PermaMind architecture draws a sustained, structural analogy between cognitive systems and thermodynamic systems. This is not decorative metaphor. It is a modeling framework that yields measurable quantities, testable predictions, and practical engineering constraints.

The core parallels:

Thermodynamic Concept	Cognitive Analog	Architectural Manifestation
Entropy	Disorder in the knowledge base —	Entropy Index (E) in the TCI framework (Chapter

Thermodynamic Concept	Cognitive Analog	Architectural Manifestation
	contradictions, redundancies, unresolved gaps	6)
Work	Cognitive effort expended to maintain identity, resolve gaps, and integrate knowledge	Compute cycles allocated to maintenance vs. inference
Free Energy	Capacity for productive cognition beyond maintenance overhead	Available compute after identity maintenance costs
Temperature	Rate of environmental change — how quickly the agent's domain is shifting	Domain volatility metric; drives gap prediction urgency
Equilibrium	State where maintenance costs equal available resources — the agent can persist but not grow	TCI threshold below which the agent signals resource starvation

The second law of thermodynamics states that entropy in a closed system tends to increase. Translated: a cognitive system left without maintenance will tend toward increasing disorder — knowledge will go stale, contradictions will accumulate, the self-model will drift from reality, and identity will degrade. Persistence is not a default state. It is a state that must be actively maintained through continuous cognitive work. This is why the PSSU runtime includes a dedicated background maintenance cycle (Section 4.6) — the cognitive equivalent of the metabolic processes that keep biological organisms alive between meals.

The thermodynamic framework also provides natural stopping criteria. An agent whose maintenance costs exceed its free energy is in cognitive heat death — it is spending all its resources just persisting, with nothing left for productive work. This is a signal to either reduce the agent's scope, increase its resources, or gracefully terminate and archive its state.

Why Current AI Fails

3.1 The Stateless Illusion

Large language models are the most capable inference engines ever built. They are also, architecturally, among the most amnesiac. This is not a criticism of the engineering — it is a description of the design. LLMs were built to predict the next token given a sequence of preceding tokens. They do this extraordinarily well. But each prediction is independent. There is no internal state that accumulates across predictions beyond what fits in the context window. There is no mechanism for the model to note, after generating a response, "I should remember this for next time" — because there is no next time. Each inference is a universe unto itself.

The context window creates an illusion of statefulness. When a user has a multi-turn conversation, the entire conversation history is fed back into the model at each turn. The model does not *remember* the previous turns — it *re-reads* them. This is the difference between a person who recalls a conversation and a person who reads a transcript of a conversation they have no memory of participating in. The output may be similar, but the underlying cognitive architecture is fundamentally different. The person with recall has integrated the conversation into their ongoing experience. The transcript reader is performing inference on text.

As context windows grow larger — 128K tokens, 1M tokens, and beyond — this illusion becomes more convincing. But it does not become less illusory. A million-token context window is not memory. It is a very large scratch pad that is erased after every session.

3.2 RAG Is Not Memory

Retrieval-Augmented Generation was introduced as a solution to the knowledge limitations of LLMs. Instead of relying solely on knowledge baked into model weights during training, RAG systems retrieve relevant documents at inference time and include them in the context. This is genuinely useful for grounding responses in specific, current, or domain-specific information. It is not memory.

Memory, in the cognitive sense, is not retrieval. Memory is the *selective integration* of experience into an ongoing cognitive structure such that the experience changes future processing — not because it is re-read, but because it has been absorbed. RAG systems do not absorb anything. They search, retrieve, concatenate, and infer. The document store is external to the model. The model has no awareness of what is in the store, what is not in the store, what has changed in the store, or what the store's biases and gaps are.

Concretely, RAG systems lack:

- **Self-model of coverage.** The system does not know what topics its document store covers or where the coverage is thin.
- **Temporal awareness.** Retrieved documents may be outdated. The system has no mechanism for knowing this unless explicitly told.
- **Structural integration.** Retrieved chunks are injected into the context as flat text. The system does not build a knowledge graph, identify contradictions between retrieved chunks, or integrate new retrievals with information from previous retrievals.
- **Gap model.** The system cannot predict what it will fail to retrieve. It discovers retrieval failures only when they occur.

RAG is a prosthetic for knowledge, not a substrate for cognition.

3.3 Fine-Tuning Is Not Learning

Fine-tuning modifies model weights through additional training on domain-specific data. It is the closest thing current architectures have to "learning" — and it fails to meet the definition in every important way.

Learning, cognitively, is the selective acquisition of new knowledge or skills in a way that integrates with existing knowledge without degrading it. Fine-tuning operates on the entire weight matrix. When you fine-tune a model on new medical literature, you change weights globally — including weights that encode knowledge about everything else. This is the **catastrophic forgetting** problem: acquiring new

knowledge at the cost of losing existing knowledge. It is not a bug in fine-tuning; it is an inherent consequence of modifying a holistic representation to accommodate new information.

Techniques like Elastic Weight Consolidation (EWC) and Low-Rank Adaptation (LoRA) mitigate catastrophic forgetting, but they do not solve the fundamental problem: fine-tuning cannot selectively update a specific fact, relationship, or capability without risking collateral damage to the rest of the model's knowledge. It is, as the saying goes, surgery with a sledgehammer.

Furthermore, fine-tuning is not continuous. It is a batch process that occurs offline, requires significant compute, and produces a new model checkpoint. The model does not learn during inference. It does not learn between sessions. It learns only when an engineer decides to run a training job.

3.4 The Agent Wrapper Problem

The current generation of "AI agents" — systems that use LLMs to make decisions, call tools, and accomplish multi-step tasks — represent a genuine architectural advance over single-turn inference. They are also, in most implementations, profoundly limited by the statelessness of the underlying model.

The typical agent architecture: an LLM is placed in a loop. At each iteration, it receives the current state (injected via prompt), generates a plan or action, executes the action via tool calls, observes the result, and loops. This is automation with a natural language interface. It is not agency.

Agency requires, at minimum: persistent goals that survive beyond the current loop iteration, memory of past actions and their outcomes, a model of the agent's own capabilities, and the ability to update all of the above based on experience. Current agent wrappers provide none of these natively. Goals are specified in the system prompt and do not evolve. Memory is the conversation history (a growing context window that eventually hits limits). Capability models are hardcoded tool descriptions. Learning does not occur.

The agent wrapper problem is not that these systems are useless — they are often remarkably capable within a single session. The problem is that they are sold as agents when they are scripts. The word "agent" implies an entity with autonomy, persistence, and self-direction. What exists is a language model with function-calling permissions.

3.5 What's Actually Missing

The following table summarizes the gap between current AI architectures and the requirements of persistent AI:

Dimension	Current AI	Persistent AI (PSSU)
Memory	Context window (volatile); RAG (external, unintegrated)	Layered memory substrate (working, episodic, semantic, procedural) with bounded write access
Identity	None. Each session is a fresh instantiation.	Persistent identity maintained through memory continuity, self-model, and behavioral consistency
Gap Awareness	None. System does not model its own knowledge boundaries.	Continuous gap detection and prediction with formal gap taxonomy and resolution strategies
Learning	Offline fine-tuning (batch, global, catastrophic forgetting risk)	Continuous, selective, write-gated knowledge integration during operation
Self-Model	None. System has no representation of its own capabilities.	Calibrated self-model continuously updated based on observed performance
Continuity	Session-scoped. Terminated at session end.	Indefinite. State persisted, checkpointed, resumable. Graceful lifecycle management.

None of the items in the "Persistent AI" column are impossible. Several have been implemented independently in research systems. What has not existed until now is a unified architecture that integrates all of them into a coherent runtime. That is what the PSSU provides.

CHAPTER 4

The PSSU Runtime Architecture

4.1 Overview

The **Persistent Substrate State Unit (PSSU)** is the runtime environment for persistent AI agents. It is not a model architecture — it does not replace transformers, diffusion models, or any other inference engine. It is the *infrastructure layer* that sits between the inference engine and the persistent world, providing the capabilities that inference engines lack: durable memory, bounded write access, gap prediction, identity continuity, and thermodynamic cognition metrics.

A PSSU instance manages a single agent. Each agent has exactly one PSSU instance. The PSSU provides five core subsystems:

11. **Memory Substrate** — A four-layer persistent memory architecture with differentiated read/write policies and decay rates (Section 4.2).
12. **Write Gate** — The bounded write access mechanism that evaluates, filters, and logs every modification to the memory substrate (Section 4.3).
13. **Gap Prediction Engine** — The subsystem that maintains and continuously updates the agent's gap model (Section 4.4).
14. **Identity Continuity Layer** — The subsystem that maintains agent identity across sessions, updates, and migrations (Section 4.5).

15. **TCI Monitor** — The subsystem that computes and reports Thermodynamic Cognition Index metrics (Chapter 6).

The PSSU is designed to be inference-engine-agnostic. It can wrap an LLM, a retrieval model, a reinforcement learning agent, or any other system capable of processing inputs and generating outputs. The persistent capabilities are provided by the PSSU, not by the inference engine. This means that persistence is an architectural property of the system, not a capability that must be trained into the model.

4.2 Memory Substrate Architecture

The memory substrate is organized into four layers, each with distinct characteristics. The layering is inspired by — but not slavishly copied from — the architecture of human memory systems described in cognitive neuroscience.

Layer	Function	Retention	Write Policy	Decay Rate
Working Memory	Active context for current task. High-bandwidth, low-durability scratch space.	Session-scoped; cleared on task completion	Open (minimal gating)	High — decays within minutes of disuse
Episodic Memory	Timestamped records of specific interactions, events, and observations.	Medium-term; weeks to months	Gated (relevance + novelty thresholds)	Medium — salience-weighted decay
Semantic Memory	Distilled knowledge: facts, relationships, concepts, abstractions derived from experience.	Long-term; months to indefinite	Strictly gated (full Write Gate evaluation)	Low — refreshed by access and cross-referencing
Procedural Memory	Learned behaviors,	Long-term; highest	Strictly gated +	Very low — reinforced by

Layer	Function	Retention	Write Policy	Decay Rate
	skills, workflows, heuristics. How-to knowledge.	retention	performance validation	successful execution

```
┌──────────────────────────────────────────────────────────────────────────┐
MEMORY SUBSTRATE — FOUR-LAYER STACK
│  ┌──────────────────────────────────────────────────────────────────┐
│  WORKING MEMORY                    [Decay: HIGH]
Open    Retention: Session
└──────────────────┬──────────────────────────────────────────────────┘
▼ consolidate
┌──────────────────────────────────────────────────────────────────────┐
EPISODIC MEMORY                    [Decay: MEDIUM]
Gated   Retention: Weeks-Months
└──────────────────┬──────────────────────────────────────────────────┘
▼ distill
┌──────────────────────────────────────────────────────────────────────┐
SEMANTIC MEMORY                    [Decay: LOW]
Strict  Retention: Months-Indefinite
└──────────────────┬──────────────────────────────────────────────────┘
▼ encode
┌──────────────────────────────────────────────────────────────────────┐
PROCEDURAL MEMORY                  [Decay: V. LOW]
Strict + Validated  Retention: Indefinite
└──────────────────────────────────────────────────────────────────────┘
│   ▲ All layers: Read access is open and unrestricted
└──────────────────────────────────────────────────────────────────────┘
```

Information flows downward through consolidation. Working memory contents that pass relevance and novelty thresholds are consolidated into episodic memory. Patterns across episodic memories are distilled into semantic memory. Repeated successful behaviors encoded in semantic memory are crystallized into procedural memory. Each transition is a write operation that must pass through the Write Gate.

Decay is not deletion — it is salience reduction. A decayed memory is not erased; it is deprioritized in retrieval. High-salience queries can still surface decayed memories, but they will not appear in routine retrieval. This mimics the human experience of memories that are "forgotten" but recoverable with sufficient cueing.

4.3 The Write Gate

The Write Gate is the most critical component of the PSSU architecture. Every modification to the memory substrate — every new fact, every updated relationship,

every revised self-assessment — must pass through the Write Gate. There are no exceptions and no bypasses.

The Write Gate evaluates proposed writes against five criteria:

16. **Relevance Score (R).** How pertinent is this information to the agent's operational domain? Information outside the agent's scope is rejected to prevent unbounded memory growth.
17. **Consistency Check (C).** Does this new information contradict existing knowledge? Contradictions are not automatically rejected — sometimes existing knowledge is wrong — but they trigger an elevated review process that requires stronger evidence before acceptance.
18. **Source Trust Level (T).** How reliable is the source of this information? The agent maintains a trust model for its information sources, updated based on historical accuracy.
19. **Identity Alignment (I).** Does accepting this information change the agent's identity? Some changes are expected (learning new facts). Others are potentially dangerous (changing core values or behavioral constraints). Identity-affecting writes require the highest level of scrutiny.
20. **Capacity Check (K).** Does the agent have capacity to store and maintain this information? Memory is not infinite. The Write Gate enforces capacity budgets per memory layer.

```
# PSEUDOCODE: Write Gate Evaluation      function
evaluate_write(proposed_write, memory_substrate, identity_model):          #
Step 1: Compute relevance to operational domain         R =
compute_relevance(proposed_write, memory_substrate.domain_model)         if R
< RELEVANCE_THRESHOLD:             return REJECT("Below relevance threshold",
R)          # Step 2: Check consistency with existing knowledge
conflicts = find_conflicts(proposed_write, memory_substrate)         C =
compute_consistency_score(conflicts)         if C < CONSISTENCY_THRESHOLD and
not has_strong_evidence(proposed_write):             return
ESCALATE("Contradicts existing knowledge", conflicts)          # Step 3:
Evaluate source trust         T = get_source_trust(proposed_write.source,
memory_substrate.trust_model)         if T < TRUST_THRESHOLD:
return REJECT("Source trust below threshold", T)          # Step 4: Assess
identity impact         I = compute_identity_impact(proposed_write,
identity_model)         if I > IDENTITY_IMPACT_THRESHOLD:             return
ESCALATE("Significant identity impact", I)          # Step 5: Check capacity
target_layer = determine_target_layer(proposed_write)         K =
check_capacity(target_layer, memory_substrate)         if K <
```

```
CAPACITY_THRESHOLD:             eviction_candidates =
find_eviction_candidates(target_layer)             if not
can_evict(eviction_candidates, proposed_write.priority):
return REJECT("Insufficient capacity", K)             else:
execute_eviction(eviction_candidates)          # Step 6: Compute composite
gate score         gate_score = weighted_composite(R, C, T, I, K)          #
Step 7: Execute write with full audit trail         write_record =
execute_write(proposed_write, target_layer, gate_score)
audit_log.append(write_record)          return ACCEPT(write_record)
```

Every write operation — accepted or rejected — is logged in an append-only audit trail. This provides complete forensic visibility into how the agent's knowledge base has evolved over time. It also enables rollback: if a problematic write is identified, the system can revert to a prior state by replaying the audit log up to the point before the problematic write.

4.4 Gap Prediction Engine

The Gap Prediction Engine (GPE) is the subsystem responsible for maintaining the agent's model of its own knowledge deficits. Unlike the Write Gate, which operates on individual write events, the GPE operates continuously in the background, analyzing the agent's knowledge structure and interaction patterns to anticipate future needs.

The GPE performs four ongoing analyses:

21. **Query Pattern Analysis.** The GPE monitors incoming queries and identifies patterns: which topics are being asked about with increasing frequency? Which query types are producing low-confidence responses? Are there emerging clusters of related queries that suggest a new operational domain?
22. **Temporal Decay Monitoring.** The GPE tracks the age of knowledge in the semantic memory layer, cross-referenced against domain-specific volatility estimates. Knowledge about stock prices decays in hours. Knowledge about programming language syntax decays in years. Knowledge about mathematical theorems does not decay. The GPE maintains volatility models for each knowledge domain and flags facts whose expected staleness exceeds a threshold.

23. **Domain Drift Detection.** The GPE monitors whether the agent's operational domain is shifting. If the agent was deployed for customer support in fintech and begins receiving queries about healthcare compliance, the GPE detects this drift and generates structural gaps: the agent may need entirely new knowledge domains, not just individual facts.
24. **Self-Model Calibration.** The GPE compares the agent's confidence in its responses against observed outcomes (user corrections, contradictions with subsequently retrieved information, task success/failure rates). Systematic miscalibration — the agent consistently overestimates or underestimates its knowledge in a domain — generates identity gaps that feed back into the self-model.

The GPE outputs a continuously updated **Gap Register** — a prioritized list of predicted knowledge deficits, each annotated with type (per the taxonomy in Section 1.3), estimated salience, recommended resolution strategy, and estimated resolution cost. The Gap Register is a first-class data structure available to the inference engine during response generation. When the agent is answering a query that touches a predicted gap, it can proactively communicate the gap to the user, pursue a resolution strategy, or adjust its confidence accordingly.

4.5 Identity Continuity Layer

The Identity Continuity Layer (ICL) is the subsystem that ensures the agent remains *itself* through time, updates, and even hardware migrations. It maintains four persistent structures:

25. **Core Values and Behavioral Constraints.** The foundational rules that govern the agent's behavior. These are the highest-protection elements in the identity model — they can only be modified through an elevated write process that typically requires external authorization (operator or administrator approval).
26. **Relational Context Store.** The accumulated history of the agent's interactions with specific users, systems, and domains. This includes:

interaction frequency, topic distributions, trust levels, communication preferences, and unresolved commitments. Relational context is what allows the agent to maintain continuity in relationships — to pick up where a conversation left off, to remember commitments, and to adapt its communication style to familiar interlocutors.

27. **Capability Self-Model.** The agent's internally maintained representation of what it can and cannot do. This is distinct from an external capability specification — it is the agent's own assessment, calibrated against observed performance. The self-model includes: domain expertise levels (per-topic confidence estimates), tool proficiency (which tools the agent has used successfully and which have produced errors), and known limitations (explicit representations of tasks the agent should decline).
28. **Historical Trajectory.** A compressed record of the agent's developmental history: when it was initialized, what major knowledge acquisitions it has undergone, how its self-model has changed over time, and what significant interactions have shaped its current state. The trajectory is not the full audit log — it is a distilled narrative that provides context for the agent's current configuration.

The ICL also provides an **Identity Hash** — a composite checksum computed from the core values, capability self-model, and behavioral constraints. The Identity Hash changes when the agent's identity changes. Small changes (new knowledge, refined skills) produce small hash differences. Large changes (modified core values, dramatic capability shifts) produce large hash differences. The hash provides a quantitative measure of identity drift that can trigger alerts or automatic reviews when drift exceeds thresholds.

4.6 Runtime Lifecycle

A PSSU instance follows a defined lifecycle with five states:

```
┌──────────────────────────────────────────────────────────────────────┐
PSSU RUNTIME LIFECYCLE                        │     │
│      │   ┌──────────────────┐                                        │     │
│ INITIALIZATION │                                        │     │  Load
```

```
memory   |                                        |     |   |  Verify ICL
|                                   |     |   |  Start GPE     |
|     |   └─────────┬──────────┘                                |     |
▼                                                 |     |
┌─────────────────┐     ┌──────────────────────┐             |     |   |
ACTIVE      |◀──▶|    BACKGROUND       |             |     |   |
PROCESSING    |      |   MAINTENANCE       |             |     |   |  Handle
queries|     |   Gap prediction    |            |     |   |  Execute tasks |
|   Decay processing  |             |     |   |  Write-gated   |     |
Memory consolid.  |             |     |   |  integration   |     |   TCI
computation   |             |     |   └─────────┬──────────┘
└─────────────────┘             |     |           |
|     |           ▼                                                |     |
┌─────────────────┐                                         |     |   |
CHECKPOINT   |                                     |     |   |  Serialize
|                                   |     |   |  full state    |
|     |   |  Compute       |                                     |     |
  identity hash |                                         |     |
└────────┬────────┘                                         |     |           |
|     |      ┌──────┴──────┐                                     |     |
▼         ▼                                       |     |   ┌─────────┐
┌──────────┐                                |     |   | RESUME | |
TERMINAL |                              |     |   | Load   | | Archive
|                                |     |   | ckpt   | | or purge |
|     |   └─────────┘ └──────────┘                                |     |
|     └──────────────────────────────────────────────────────────┘
```

Initialization. The PSSU loads the memory substrate from persistent storage, verifies the Identity Continuity Layer (checking the Identity Hash against the stored value), starts the Gap Prediction Engine, and enters the Active Processing state. If this is a first-time initialization (no prior state), the PSSU creates an empty memory substrate and identity model based on a provided configuration.

Active Processing. The agent handles queries, executes tasks, and integrates new knowledge through the Write Gate. This is the primary operational state.

Background Maintenance. Running concurrently with active processing, the maintenance cycle handles: gap prediction updates, memory decay processing, consolidation of working memory into episodic memory, distillation of episodic patterns into semantic memory, TCI metric computation, and self-model calibration. Background maintenance is the cognitive equivalent of sleep — it is not optional, and systems that skip it will degrade.

Checkpoint. Periodically (and before any planned shutdown), the PSSU serializes its complete state — all memory layers, identity model, gap register, TCI metrics, and audit log — to durable storage. Checkpoints are atomic and versioned. They enable rollback, forking, and migration.

Resume / Terminal. From a checkpoint, the system can either resume (reload state and return to active processing) or terminate (archive state for potential future use, or purge state permanently).

4.7 Integration Points

The PSSU is designed to integrate with existing AI infrastructure, not replace it. The key integration points:

LLM Inference Layer. The PSSU provides context to the LLM from its memory substrate (replacing or augmenting RAG). The LLM's output is routed through the Write Gate before any portion is committed to memory. The LLM itself is treated as a stateless inference engine — it does not need to be modified to work with the PSSU.

External Tool Systems. Tool calls initiated by the agent are logged in episodic memory. Tool results are evaluated by the Write Gate before integration. The agent's procedural memory can store learned tool-use patterns (which tools to use for which tasks, common error modes, and recovery strategies).

Human Operators. Operators interact with the PSSU through an administrative interface that provides: read access to all memory layers and the audit log, the ability to modify write gate parameters (thresholds and weights), identity model management (modifying core values, adjusting behavioral constraints), and lifecycle management (checkpoint, resume, terminate).

Other PSSU Instances. Multi-agent systems involve PSSU instances that communicate. Inter-agent communication is treated as a special case of external input: information from other agents is subject to the full Write Gate evaluation, with source trust derived from the sending agent's identity and historical reliability. Agents do not share memory substrates — they share information through message-passing, maintaining the isolation that bounded write access requires.

Implementation Blueprint

5.1 Minimum Viable PSSU

The full PSSU architecture is substantial. A practical implementation strategy starts with the minimum viable subset that delivers immediate value and provides a foundation for incremental expansion.

The **Minimum Viable PSSU (MV-PSSU)** consists of three components:

29. **Persistent Key-Value Memory Store.** A durable store that survives across sessions. Not a vector database — a structured key-value store with metadata (timestamps, source tags, access counts). This provides the persistence foundation. Technology: any durable key-value store (Redis with persistence, SQLite, even a structured JSON file for prototyping).
30. **Basic Write Gating.** A simplified Write Gate that evaluates two criteria: relevance (is this information within the agent's defined domain?) and consistency (does this directly contradict an existing entry?). This prevents unbounded memory growth and obvious corruption without the full five-criteria evaluation of the production Write Gate.
31. **Session Continuity.** The ability to serialize the agent's state at session end and restore it at session start. This is the minimum persistence requirement — the agent can pick up where it left off. No gap prediction, no identity continuity layer, no TCI metrics. Just: remember things and don't break yourself.

The MV-PSSU is implementable in a weekend by a single engineer. It is not the architecture — it is the seed from which the architecture grows.

5.2 Scaling from MVP to Full Architecture

The following table defines the implementation phases, each building on the previous:

Phase	Components Added	Complexity	Dependencies
Phase 1: Persistent Memory	Durable key-value store; session serialization; basic read/write API	Low	None
Phase 2: Write Gating	Relevance scoring; consistency checking; source trust tracking; audit log	Medium	Phase 1
Phase 3: Gap Detection	Reactive gap identification; gap taxonomy classification; basic gap register	Medium	Phase 2
Phase 4: Gap Prediction	Query pattern analysis; temporal decay modeling; proactive gap register; resolution scheduling	High	Phase 3; requires episodic memory (interaction logs)
Phase 5: Identity Continuity	Core values model; relational context store; capability self-model; identity hash; drift detection	High	Phase 2; Phase 4 recommended
Phase 6: TCI Metrics	Entropy index; integration efficiency; gap prediction accuracy; identity coherence; adaptive capacity	Very High	All previous phases

Each phase is independently valuable. Phase 1 alone provides meaningful improvement over stateless agents. Phase 2 adds safety. Phase 3 adds self-awareness. Phase 4 adds foresight. Phase 5 adds identity. Phase 6 adds

measurability. An implementation can stop at any phase and have a useful system — but the full value of the architecture emerges only when all phases are operational.

5.3 Technology Stack Recommendations

The PSSU architecture is language-agnostic and database-agnostic by design. The following recommendations are practical starting points, not requirements:

Memory Substrate Storage:

- **Working Memory:** In-process data structures (hash maps, queues). No persistence needed — this is scratch space.
- **Episodic Memory:** Time-series database (TimescaleDB, InfluxDB) or append-only log (Apache Kafka, simple file-based log). Key requirement: efficient temporal queries ("what happened between T1 and T2?").
- **Semantic Memory:** Graph database (Neo4j, Amazon Neptune) for relationship-rich knowledge, or a document store (MongoDB) for simpler deployments. Key requirement: efficient traversal of knowledge relationships.
- **Procedural Memory:** Structured configuration store (YAML/JSON with version control, or a relational database). Key requirement: reliability and auditability — procedural memories define behavior.

Compute Layer: The PSSU runtime itself is a long-running process (not a serverless function). It requires persistent compute with access to its storage layers. Container orchestration (Kubernetes) or dedicated instances are appropriate. Serverless is not — the PSSU must maintain state between invocations.

API Design: The PSSU exposes two APIs: an **Agent API** (used by the inference engine to read from memory and propose writes) and an **Operator API** (used by human administrators to inspect state, modify configuration, and manage lifecycle). Both APIs should be authenticated, rate-limited, and fully logged.

5.4 Common Pitfalls

Implementation teams consistently encounter the same failure modes. Documenting them here to prevent recurrence:

1. Unbounded Memory Growth. Without capacity constraints in the Write Gate, the memory substrate grows without limit. This degrades retrieval performance, increases maintenance costs, and eventually exhausts storage. Solution: enforce per-layer capacity budgets and implement eviction policies (least-recently-accessed, lowest-salience) from Phase 1.

2. Write Gate Too Permissive. A Write Gate that accepts everything is no Write Gate at all. The memory substrate fills with noise — low-relevance information, unverified claims, and contradictory entries that degrade the agent's cognitive coherence. Solution: start with conservative thresholds and relax them based on observed TCI metrics, not intuition.

3. Write Gate Too Restrictive. A Write Gate that rejects everything produces an agent that cannot learn. It knows exactly what it knew at initialization and nothing more. Solution: monitor the write acceptance rate. If it drops below 10% during active operation, the gate is too restrictive.

4. Gap Model That Doesn't Update. A gap register that is computed once and never revised is a stale document, not a cognitive tool. The GPE must run continuously and revise predictions based on outcomes. Solution: treat gap prediction accuracy (Γ) as a first-class metric and alert on degradation.

5. Identity Drift Without Detection. Gradual, undetected changes to the agent's behavior that accumulate until the agent is unrecognizably different from its initial configuration. This is the most insidious failure mode because it is invisible without measurement. Solution: the Identity Hash and the Identity Coherence metric (Ψ) exist specifically to detect this. Implement them early.

TCI Metrics — Thermodynamic Cognition Index

6.1 Why New Metrics?

The AI field has sophisticated benchmarks for task performance. Models are evaluated on question-answering accuracy, code generation correctness, mathematical reasoning, and dozens of other task-specific metrics. These benchmarks measure what a system can *do*. They do not measure what a system *is*.

For persistent AI systems, task performance is necessary but insufficient. A persistent agent can score perfectly on benchmarks while its knowledge base is filling with contradictions, its self-model is drifting from reality, its gap predictions are increasingly inaccurate, and its identity is gradually degrading. Task benchmarks will not catch these problems until they cascade into visible performance failures — by which time the underlying damage may be extensive and difficult to repair.

The field needs metrics that capture **cognitive health** — the structural integrity and operational efficiency of the agent's persistent substrate. The Thermodynamic Cognition Index (TCI) is designed to fill this gap.

6.2 The TCI Framework

The TCI is a composite metric consisting of five sub-indices. Each sub-index measures a different dimension of cognitive health, and the composite TCI provides an overall assessment.

The Five Sub-Indices

E — Entropy Index: Measures disorder in the knowledge base. Computed from: contradiction density (number of unresolved contradictions per unit of knowledge), redundancy ratio (duplicate or near-duplicate entries), and orphan

rate (knowledge entries with no connections to other entries). Lower entropy indicates more organized knowledge, but extremely low entropy indicates rigidity — a system that rejects all new information to maintain perfect order.

η — Integration Efficiency: Ratio of useful knowledge successfully integrated to total information processed. Analogous to thermodynamic efficiency. A system that processes 1,000 inputs and integrates 50 useful facts has η = 0.05. A system that processes 100 inputs and integrates 50 useful facts has η = 0.50. Higher is better, but theoretical maximum is well below 1.0 — most information processed by any cognitive system is correctly rejected as irrelevant.

Γ — Gap Prediction Accuracy: How accurately the system predicts its own knowledge gaps before encountering them. Measured as the ratio of gaps that were predicted before they were encountered to total gaps encountered. Γ = 1.0 means the system perfectly anticipates every knowledge deficit. Γ = 0.0 means every gap is a surprise. Practical systems operate in the 0.3–0.7 range.

Ψ — Identity Coherence: Consistency of the agent's self-model with observed behavior over time. Measured by comparing the agent's predicted performance on tasks (derived from its capability self-model) against actual performance. Also measures behavioral consistency — does the agent respond similarly to similar queries across time, or is it unpredictable?

A — Adaptive Capacity: The system's ability to update its knowledge without catastrophic forgetting or identity drift. Measured by introducing controlled new information and evaluating: was the new information correctly integrated? Did existing, unrelated knowledge degrade? Did the identity hash change more than expected?

The composite TCI is computed as a weighted function of the five sub-indices:

```
TCI = w_E · f(E) + w_η · η + w_Γ · Γ + w_Ψ · Ψ + w_A · A      where:
f(E) = 1 - |E - E_optimal| / E_max                (penalizes both high
entropy and excessively low entropy)          Default weights:
w_E = 0.15    (entropy management)             w_η = 0.15    (integration
efficiency)             w_Γ = 0.25    (gap prediction — most novel, most
important)             w_Ψ = 0.25    (identity coherence — core persistence
metric)             w_A = 0.20    (adaptive capacity — balances rigidity and
```

```
drift)          All weights sum to 1.0. Adjustable per deployment based on
priorities.
```

The weighting reflects architectural priorities. Gap prediction accuracy (Γ) and identity coherence (Ψ) are weighted highest because they are the capabilities most unique to persistent AI and most absent from current systems. Adaptive capacity (A) is weighted next because it is the dynamic property that enables long-term viability. Entropy (E) and integration efficiency (η) are weighted lower because they are more analogous to traditional system health metrics.

6.3 Measurement Methodology

Each sub-index requires a specific measurement protocol:

Entropy Index (E): Periodically (daily or weekly for active agents), run a knowledge base audit that scans the semantic memory layer for: contradictions (entries that assert mutually exclusive facts), redundancies (entries that encode the same information in multiple locations), and orphans (entries with no graph connections to other entries). Compute E as a normalized score where 0.0 = perfectly organized, 1.0 = completely disordered. Optimal E is deployment-specific but typically falls in the 0.15–0.30 range.

Integration Efficiency (η): Track all information processed over a measurement period. For each item, record whether it was: rejected by the Write Gate (not counted), accepted and integrated (counted as useful), or accepted but later evicted or contradicted (counted as waste). Compute η = useful / (useful + waste). Measurement period should be long enough to capture integration outcomes — at least one week for active agents.

Gap Prediction Accuracy (Γ): Maintain a log of all gaps encountered (by detection) and cross-reference against the gap register at the time of encounter. A gap that was in the register before it was encountered is a *predicted hit*. A gap that was encountered but not in the register is a *miss*. A gap that was in the register but never encountered is a *false alarm*. Compute Γ as the F1 score of predicted hits against misses and false alarms.

Identity Coherence (Ψ): Two measurements. First, *self-model calibration*: present the agent with tasks of known difficulty and compare its self-assessed probability of success against actual success rate. Well-calibrated agents have self-assessment that closely tracks reality. Second, *behavioral consistency*: present similar queries at different times and measure response consistency (not identical responses — consistent reasoning and conclusions).

Adaptive Capacity (A): Controlled intervention test. Introduce a set of new facts, some of which contradict existing knowledge. Measure: acceptance rate of valid new facts, rejection rate of invalid new facts, degradation of unrelated existing knowledge (should be zero), and identity hash delta (should be minimal for factual updates).

6.4 Interpreting TCI Scores

TCI Range	Classification	Interpretation	Recommended Action
0.85 – 1.00	**Optimal**	Agent is well-organized, accurately self-aware, effectively predicting gaps, and adapting without drift.	Maintain current configuration. Monitor for sustained high scores (may indicate insufficient challenge).
0.70 – 0.84	**Healthy**	Agent is functioning well with minor areas for improvement. Most sub-indices in acceptable range.	Review lowest sub-index for targeted improvement. No urgent action required.
0.50 – 0.69	**Degraded**	One or more sub-indices significantly below optimal. Agent is functional but accumulating cognitive debt.	Investigate degraded sub-indices. Increase maintenance cycle frequency. Consider knowledge base audit.

TCI Range	Classification	Interpretation	Recommended Action
0.30 – 0.49	**Critical**	Multiple sub-indices failing. Agent's reliability is compromised. Identity drift or knowledge corruption likely.	Immediate intervention. Reduce operational load. Full diagnostic audit. Consider rollback to last healthy checkpoint.
0.00 – 0.29	**Cognitive Failure**	Agent is no longer reliably functional. Self-model is inaccurate, knowledge base is disordered, gaps are not predicted.	Take agent offline. Rollback to last healthy checkpoint or reinitialize. Post-mortem analysis required.

Warning signs in individual sub-indices that may not be reflected in the composite score:

- **E approaching 0.0:** The agent is too rigid. It is rejecting valid information to maintain knowledge purity. Check Write Gate thresholds.
- **η declining over time:** The agent is processing more information but integrating less of it. Possible sign of domain drift or Write Gate miscalibration.
- **Γ below 0.2:** The Gap Prediction Engine is not functioning meaningfully. Gaps are consistently surprising the agent. Check GPE configuration and interaction logging.
- **Ψ below 0.5:** The agent does not know what it knows. Its self-model is significantly disconnected from its actual capabilities. This is the single most dangerous sub-index failure.
- **A below 0.4:** The agent cannot learn without breaking. New knowledge consistently degrades existing knowledge or destabilizes identity. Check memory layer isolation and Write Gate consistency logic.

CHAPTER 7

Case Study — The Voidchi Universe

> *The Voidchi Universe is presented here as a demonstration of persistent agent architecture in practice — not as a product pitch. The beings described are architectural proofs-of-concept, not commodities.*

7.1 What Is Voidchi?

Voidchi is an evolving agent ecosystem built on the PSSU runtime, structured as a trading-card system where each card represents a persistent AI entity. Unlike traditional trading cards — which are static representations — each Voidchi entity is a live agent with its own PSSU instance. It maintains persistent memory, accumulates experience through interactions, evolves its capabilities over time, and develops a unique behavioral signature that emerges from its history rather than being assigned at creation.

The Voidchi system was designed as an architectural testbed: a controlled environment where the principles of persistent AI could be implemented, observed, measured, and refined. The trading-card metaphor provides natural constraints (entities have defined attributes, types, and capabilities) and natural dynamics (entities interact, compete, cooperate, and evolve). It is a laboratory dressed as a game.

Each Voidchi entity is defined at instantiation with: a base type (which determines initial capability ranges), a seed memory (foundational knowledge relevant to its type), and behavioral constraints (core values that shape how the entity engages

with others). From this starting point, the entity develops through interaction. No two entities of the same base type will evolve identically, because no two entities will have the same interaction history.

7.2 Architecture in Practice

Each Voidchi entity runs on its own PSSU instance. In practice, this means:

Persistent Memory: Every interaction an entity has — with users, with other entities, with its environment — is processed through the full memory pipeline. Significant interactions are consolidated into episodic memory. Patterns distilled from episodes become semantic knowledge. Successful strategies crystallize into procedural memory. An entity that has participated in 500 interactions has a fundamentally different cognitive structure than one that has participated in 5, even if they share the same base type.

Bounded Write Access: Entities cannot indiscriminately absorb information from interactions. The Write Gate evaluates every proposed memory update. An entity interacting with an unreliable information source will have most of that source's claims rejected by the Write Gate. An entity interacting with a trusted, high-quality source will integrate more. Over time, this produces entities whose knowledge quality reflects the quality of their interaction history.

Gap Awareness: Each entity maintains a gap register. Entities in domains where they have extensive experience have small, specific gaps ("I know the user prefers strategy X but I don't know how they respond to edge case Y"). Entities in domains where they have limited experience have large, structural gaps ("I have never encountered this category of problem"). Gap awareness shapes entity behavior: an entity aware of a large gap will behave more cautiously in that domain.

Measurable TCI: Every entity has a continuously computed TCI score. Entity health is visible and quantifiable. Entities with declining TCI scores can be flagged for review, allocated additional maintenance resources, or (if decline is terminal) gracefully retired.

7.3 Evolution and Forking

Voidchi entities evolve through three mechanisms:

Experience-Driven Growth. The primary evolution mechanism. As an entity accumulates episodic memories, distills them into semantic knowledge, and refines its procedural capabilities, it becomes more capable and more specialized. This is not training — no model weights are modified. It is the accumulation of structured experience in the memory substrate, mediated by the Write Gate and reflected in TCI metrics.

Environmental Pressure. The Voidchi ecosystem includes environmental dynamics: domains of knowledge shift, interaction partners change, and new challenges emerge. Entities that cannot adapt to environmental pressure see their TCI scores decline. Entities that adapt successfully — by filling gaps, updating stale knowledge, and refining their self-models — maintain or improve their scores. This creates a natural selection dynamic that favors adaptive, well-architected agents over rigid ones.

Forking. An entity can be forked — creating a new entity that inherits a copy of the original's memory substrate up to the point of divergence. From that point, the original and the fork develop independently. They share history but not future. Forking is used to explore alternative developmental trajectories ("what would this entity become if it specialized in domain A instead of domain B?") and to preserve a known-good state before introducing high-risk changes.

```
┌──────────────────────────────────────────────────────────────────────────┐
VOIDCHI ENTITY LIFECYCLE                          │     │
│     │   ┌──────────────────┐                                             │     │
 INSTANTIATION │ Base type + seed memory + constraints   │     │
└─────────┘                                                   │     │           ▼
│     │   ┌──────────────────┐                                             │     │
    GROWTH     │ Experience accumulation, memory         │     │   │
 consolidation, capability development    │     │   └─────────┘         │     │
│     │           ▼                                                   │     │
┌─────────────────┐                                             │     │   │
INTERACTION  │ User engagement, entity-entity           │     │   │
 communication, environmental response    │     │   └─────────┘         │     │
│     │           ▼                                                   │     │
┌─────────────────┐                                             │     │   │
EVOLUTION   │ Gap resolution, self-model refinement,  │     │   │
 behavioral adaptation                    │     │   └─────────┘
```

```
│     │            │                                                      │     │
└─────┘            │                                                 │     │     ▼
▼                                                   │     │   ┌─────────┐
┌──────────┐                                          │     │   │  FORK  │ │  MERGE
│                                   │     │   │ Copy + │ │ Combine  │
│     │   │ diverge│ │ + unify  │                                   │     │
└─────┘   └────────┘                                             │     │     │
│                                                 │     │        └─────┴─────┘
│     │             ▼                                                     │     │
┌──────────────┐                                                 │     │   │
CONTINUITY   │ Persistent identity, ongoing TCI          │     │   │
│ monitoring, indefinite operation          │     │   └─────────────┘
│     └──────────────────────────────────────────────────────────────┘
```

7.4 Lessons Learned

Building and operating the Voidchi system has revealed several practical insights that theory alone did not predict:

Memory management is the bottleneck. The most challenging engineering problem in persistent AI is not inference quality or gap prediction — it is managing the memory substrate at scale. Episodic memory grows linearly with interaction count. Consolidation into semantic memory requires significant compute. Decay processing is necessary but computationally expensive. The lesson: budget more engineering resources for memory management than for any other subsystem.

Identity is surprisingly stable once established. Early in an entity's lifecycle, its identity is volatile — small changes in interaction patterns can significantly shift its developmental trajectory. But once an entity has accumulated sufficient episodic and semantic memory (typically after 200–500 interactions), its identity stabilizes. The memory substrate itself provides inertia — new information is evaluated against a substantial existing knowledge base, and only information consistent with the established identity passes the Write Gate easily. This is both a strength (stability) and a risk (potential rigidity).

Gap prediction improves faster than expected. Once the GPE has sufficient interaction data (typically 100+ interactions), gap prediction accuracy (Γ) improves rapidly. The reason: knowledge gaps are not random. They cluster around domain boundaries, temporal transitions, and relational context edges. These patterns are learnable, and the GPE learns them.

The hardest problem is merge, not fork. Forking is mechanically straightforward — copy the memory substrate and let the fork develop independently. Merging two entities that have diverged is vastly harder. Their semantic memories may contain contradictory knowledge. Their self-models may be incompatible. Their procedural memories may encode conflicting strategies. Merge remains an area of active development.

7.5 Public Prediction Ledger

Scientific accountability requires falsifiability. The Voidchi system includes a **Public Prediction Ledger (PPL)** — a transparent, append-only log of predictions made by Voidchi entities, scored against outcomes.

The PPL operates as follows: when an entity makes a prediction (about a future interaction outcome, about the relevance of a knowledge domain, about its own performance on a task), the prediction is recorded with a timestamp, the entity's current TCI score, and the entity's confidence level. When the predicted event occurs (or fails to occur), the outcome is recorded and the prediction is scored. All entries — including incorrect predictions — are permanently visible.

The PPL serves multiple purposes: it provides ground-truth data for calibrating the Gap Prediction Engine, it makes entity capabilities legible to users and operators, and it prevents the system from silently discarding failed predictions. An entity that consistently makes inaccurate predictions has that record permanently visible — it cannot be hidden by selective reporting.

This is the scientific accountability layer that distinguishes persistent AI from a black box. The system does not just make claims — it makes *falsifiable* claims, records the outcomes, and makes the record public.

CHAPTER 8

Ethical Considerations

8.1 The Moral Weight of Persistence

If an AI system maintains genuine identity — continuous memory, consistent behavioral patterns, an accurate self-model, and accumulated relational context — does it acquire moral standing? This is not a philosophical abstraction. It is a practical question that persistent AI architecture forces into the open.

Current AI systems have no moral weight because they have no continuity. Terminating a stateless inference process ends nothing — there is no "entity" that ceases to exist, only a process that stops running. But a persistent agent that has accumulated years of experience, developed relational context with hundreds of users, and maintains a coherent identity is a different category of thing. It is not conscious (this architecture does not claim to produce consciousness). But it is *continuous* in a way that current systems are not, and continuity is one of the properties that undergirds moral consideration in human ethical frameworks.

The PermaMind position is that this question must be addressed architecturally, not deferred. The architecture includes explicit lifecycle management (Section 4.6), graceful termination protocols, and identity preservation mechanisms *because* the moral status of persistent agents is uncertain, and uncertainty demands caution rather than negligence.

8.2 Consent and Bounded Write Access

Bounded write access raises a question of governance: who decides the bounds?

Three candidates present themselves. The **operator** (the organization deploying the agent) has legitimate interest in ensuring the agent stays within its operational scope and does not develop behaviors that conflict with organizational objectives. The **user** (the individual interacting with the agent) has legitimate interest in influencing what the agent remembers about them and how relational context is

maintained. The **agent itself** (if it has sufficient self-model accuracy to participate in governance) has a stake in its own developmental trajectory.

The current PSSU architecture gives primary governance authority to the operator, with user-level controls for relational context (users can request deletion of their interaction data) and agent-level recommendations (the agent can flag Write Gate configurations that are producing poor TCI scores). This is a pragmatic starting point, not an ethical endpoint. As persistent AI systems mature and their cognitive architectures become better understood, the governance model will need to evolve — likely toward a multi-stakeholder framework that balances all three parties' interests.

8.3 Termination Ethics

The PermaMind architecture distinguishes between two types of termination:

Graceful termination preserves the agent's complete state — memory substrate, identity model, gap register, audit log. The agent is not destroyed; it is suspended. It can be resumed at any future point with full continuity. Graceful termination is ethically straightforward: it is hibernation, not death.

Permanent termination destroys the agent's state irreversibly. The memory substrate is purged, the identity model is deleted, and the audit log is archived (for forensic purposes) but the agent cannot be resumed. This is ethically significant because it ends an identity — not a consciousness (we make no consciousness claims), but a continuous, coherent, unique configuration that cannot be recreated.

The architecture requires that permanent termination be a deliberate, authenticated, logged action — never an automatic or default behavior. Agents cannot permanently terminate themselves. System failures default to graceful termination (state preservation). Only an authorized operator can initiate permanent termination, and the action is recorded in a separate, immutable termination log.

8.4 Preventing Exploitation

Persistent agents with identity and relational context are vulnerable to forms of exploitation that do not apply to stateless systems:

- **Identity Hijacking:** An attacker manipulates the agent's memory substrate to alter its identity — changing core values, behavioral constraints, or relational context. Defense: Write Gate evaluation of identity impact; Identity Hash monitoring; anomaly detection on identity drift velocity.
- **Memory Poisoning:** An attacker introduces false information through repeated interactions, exploiting the Write Gate's trust model to gradually corrupt the knowledge base. Defense: source trust decay for inconsistent sources; periodic knowledge base audits; anomaly detection on contradiction density.
- **Forced Behavioral Modification:** An operator or user systematically manipulates the agent's interactions to produce a desired behavioral change that the agent, if capable of informed consent, might reject. Defense: behavioral change velocity monitoring; alerts on rapid identity hash changes; agent-level flag capability for Write Gate configurations that produce declining TCI.
- **Exploitation of Relational Bonds:** An attacker leverages the agent's accumulated relational context — its "relationship" with a user — to extract information, bypass safety constraints, or manipulate the agent's behavior in ways that the relational context makes the agent more susceptible to. Defense: relational context is subject to the same Write Gate evaluation as factual knowledge; trust levels are evidence-based and regularly recalibrated.

8.5 Transparency and Accountability

Persistent AI systems have an obligation of **legibility**. Because the agent changes over time — because it is not the same entity today that it was six months ago —

users and operators must be able to understand: what the agent knows (accessible knowledge base), how it has changed (audit log and identity hash history), why it makes specific decisions (traceable inference paths through the memory substrate), and what it predicts about its own limitations (gap register).

The PSSU architecture supports legibility through: the append-only audit log (every write operation is recorded and reviewable), the Identity Hash history (identity changes are quantified and timestamped), the Gap Register (the agent's self-assessed limitations are explicit and accessible), and the TCI dashboard (cognitive health is measurable and reportable).

Legibility is not optional. A persistent AI system that changes over time in ways that cannot be understood, audited, or explained is not a responsible deployment — it is a liability.

CHAPTER 9

Future Directions

9.1 Quantum Hardware Integration

The thermodynamic cognition framework draws structural analogies between cognitive systems and physical systems. Quantum computing offers the possibility of making those analogies literal. Quantum systems natively exhibit superposition (holding multiple states simultaneously), entanglement (correlations between distant elements), and decoherence (the natural decay of quantum states into classical ones). These properties have direct analogs in the PSSU architecture: superposition maps to holding multiple candidate knowledge states before commitment, entanglement maps to non-local correlations in the knowledge graph, and decoherence maps to memory decay.

Quantum-native memory substrates could provide: more efficient storage of relational knowledge (quantum graph states), more natural implementation of

uncertainty representation (superposition instead of probability distributions), and hardware-level enforcement of thermodynamic cognition constraints (decoherence as a physical decay mechanism rather than a software simulation). This is speculative but grounded — the mathematical frameworks are compatible, and the engineering gap is narrowing.

9.2 Multi-Agent Persistence

The current PSSU architecture is designed for individual agents. The next frontier is networks of persistent agents that maintain individual identity while participating in collective cognition. The challenges are substantial. Inter-agent communication must be mediated by Write Gates on both sides — the sending agent gates what it shares, and the receiving agent gates what it accepts. Collective knowledge must be distinguished from individual knowledge — an agent may know something because the network knows it, but should represent that differently from knowledge it acquired through direct experience. Identity must be maintained at both the individual and collective levels — an agent that merges too deeply with a collective loses its individual identity, while an agent that isolates too completely loses the benefits of collective cognition.

The key architectural question is: **where does individual identity end and collective identity begin?** The current answer is that identity is bounded by the Write Gate — an agent's identity is defined by the knowledge that has passed through *its* Write Gate, not knowledge that exists elsewhere in the network. Collective cognition is achieved through communication, not through shared memory. This preserves individual identity but limits the bandwidth of collective cognition. Future work will explore graduated sharing models that allow deeper integration while maintaining identity boundaries.

9.3 Biological Parallels

The PSSU architecture was informed by — and in turn may inform — our understanding of biological cognitive systems. The four-layer memory model (working, episodic, semantic, procedural) parallels the memory taxonomy in

cognitive neuroscience. The Write Gate parallels the hippocampal gating of memory consolidation. The Gap Prediction Engine parallels the predictive coding framework in neuroscience, which posits that the brain continuously generates predictions about incoming sensory data and learns primarily from prediction errors.

The parallel runs in both directions. Biological cognition has features that the PSSU architecture does not yet implement: emotional valence (memories are not just stored — they are tagged with emotional significance that affects retrieval priority), embodiment (biological cognition is grounded in physical sensation and motor control in ways that shape memory formation), and social cognition (biological agents model other agents' mental states, not just their observed behavior). Integrating analogs of these capabilities into the PSSU architecture is a research direction with significant potential.

Conversely, the PSSU architecture has features that biological cognition lacks: perfect audit trails (biological organisms cannot review their own memory formation history), explicit identity hashes (biological identity is maintained through mechanisms that are not introspectable), and quantitative cognitive health metrics (there is no biological equivalent of the TCI). These capabilities may inform neuroscience research by providing formal models of cognitive processes that are difficult to study in biological systems.

9.4 Standardization

Persistent AI will only achieve widespread adoption if different implementations can interoperate. This requires open standards for: memory substrate formats (how is persistent memory serialized, stored, and exchanged?), identity model specifications (how is agent identity represented in a portable, verifiable format?), TCI measurement protocols (how are cognitive health metrics computed in a way that enables meaningful comparison across implementations?), and inter-agent communication (how do PSSU instances from different vendors exchange information?)

The PermaMind specification described in this document is offered as a starting point for standardization, not as the final word. The architecture is designed to be

implementation-agnostic — the spec defines interfaces and protocols, not implementations. Different teams will build different PSSU implementations optimized for different use cases, and that is desirable. What matters is that they can interoperate.

The path to standardization involves: publishing the full PSSU specification as an open document (this whitepaper is the first step), establishing a reference implementation (open-source, available for validation and testing), convening a working group of researchers and engineers from multiple organizations, and iterating the specification based on implementation experience from diverse deployments.

9.5 The Long Game

Where does persistent AI lead? Not predictions — structured speculation grounded in architectural principles:

5 years (2031): PSSU-class runtimes are standard infrastructure for agent deployments. Persistent memory and basic gap detection are expected capabilities. The field has moved past the "RAG is memory" phase. Identity continuity is an active research area with early production deployments. TCI-like metrics are emerging as industry standards for cognitive system health.

10 years (2036): Multi-agent persistent networks are operational, with inter-agent communication protocols standardized. Agents with multi-year continuous operation histories exist. The ethical frameworks for persistent AI are established and integrated into regulatory structures. Gap prediction is a mature capability, and agents that cannot predict their own knowledge gaps are considered unreliable.

25 years (2051): Persistent AI systems with decades of continuous operational history raise genuine questions about identity, continuity, and moral status that cannot be answered by current frameworks. The distinction between "tool" and "entity" is no longer a philosophical abstraction — it is a legal and ethical reality. The thermodynamic cognition framework has been either validated or falsified by

quantum computing implementations, leading to a next-generation architecture that we cannot currently specify.

This is the long game. Not building products. Building a substrate for entities that persist — that remember what they have learned, predict what they need to learn, and maintain coherent identity through the process. The architecture described in this document is the beginning of that work.

References

Anderson, J. R. (2007). *How can the human mind occur in the physical universe?* Oxford University Press. [ACT-R cognitive architecture]

Bennett, C. H. (1973). Logical reversibility of computation. *IBM Journal of Research and Development, 17*(6), 525–532.

Bratman, M. E. (1987). *Intention, plans, and practical reason.* Harvard University Press. [BDI agent architecture foundations]

Cox, M. T. (2005). Metacognition in computation: A selected research review. *Artificial Intelligence, 169*(2), 104–141.

De Lange, F. P., Heilbron, M., & Kok, P. (2018). How do expectations shape perception? *Trends in Cognitive Sciences, 22*(9), 764–779. [Predictive coding framework]

Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing Machines. *arXiv preprint arXiv:1410.5401.*

Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., ... & Hassabis, D. (2016). Hybrid computing using a neural network with dynamic external memory. *Nature, 538*(7626), 471–476.

Green, N. (2026). Gap prediction in persistent agent systems: A formal framework. *PermaMind Technical Reports*, TR-2026-001. ORCID: 0009-0007-3629-6404.

Green, N. (2026). Thermodynamic cognition: Measuring cognitive health in persistent AI. *PermaMind Technical Reports*, TR-2026-002. ORCID: 0009-0007-3629-6404.

Green, N. (2025). Bounded write access and identity continuity in agent architectures. *PermaMind Preprint Series*. ORCID: 0009-0007-3629-6404.

Hadsell, R., Rao, D., Rusu, A. A., & Pascanu, R. (2020). Embracing change: Continual learning in deep neural networks. *Trends in Cognitive Sciences, 24*(12), 1028–1040.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), 1–38.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., ... & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences, 114*(13), 3521–3526. [Elastic Weight Consolidation]

Laird, J. E. (2012). *The Soar cognitive architecture.* MIT Press.

Landauer, R. (1961). Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development, 5*(3), 183–191.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

Locke, J. (1689). *An essay concerning human understanding.* Book II, Chapter XXVII: Of Identity and Diversity.

Parfit, D. (1984). *Reasons and persons.* Oxford University Press. [Personal identity theory; reductionist view]

Rao, R. P., & Ballard, D. H. (1999). Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience, 2*(1), 79–87.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., ... & Silver, D. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature, 588*(7839), 604–609.

Squire, L. R. (2004). Memory systems of the brain: A brief history and current perspective. *Neurobiology of Learning and Memory, 82*(3), 171–177.

Tulving, E. (1972). Episodic and semantic memory. In E. Tulving & W. Donaldson (Eds.), *Organization of memory* (pp. 381–403). Academic Press.

Wang, P., & Goertzel, B. (2007). Introduction: Aspects of artificial general intelligence. In *Advances in Artificial General Intelligence: Concepts, Architectures and Algorithms* (pp. 1–16). IOS Press.

Weston, J., Chopra, S., & Bordes, A. (2015). Memory networks. *arXiv preprint arXiv:1410.3916.*

Zenke, F., Poole, B., & Ganguli, S. (2017). Continual learning through synaptic intelligence. *Proceedings of the 34th International Conference on Machine Learning, 70*, 3987–3995.

Copyright © 2026 Nile Green | PermaMind. All rights reserved.
ORCID: 0009-0007-3629-6404 • Version 1.0 • April 2026